

EfficientZerov2 with JAX – Build Log 2

I. MCTX 트리구조 분석하기

II. 새로 만들기

III. 간단 테스트

I. MCTX 트리구조 분석하기

```
@chex.dataclass(frozen=True)
class Tree(Generic[T]):
    """State of a search tree.
    node_visits: chex.Array # [B, N]
    raw_values: chex.Array # [B, N]
    node_values: chex.Array # [B, N]
    parents: chex.Array # [B, N]
    action_from_parent: chex.Array # [B, N]
    children_index: chex.Array # [B, N, num_actions]
    children_prior_logits: chex.Array # [B, N, num_actions]
    children_visits: chex.Array # [B, N, num_actions]
    children_rewards: chex.Array # [B, N, num_actions]
    children_discounts: chex.Array # [B, N, num_actions]
    children_values: chex.Array # [B, N, num_actions]
    embeddings: Any # [B, N, ...]
    root_invalid_actions: chex.Array # [B, num_actions]
    extra_data: T # [B, ...]
```

B : Batch Size

N : Node

개수

B : Batch Size

N : Node 개수

num_actions: 액션 수

B : Batch Size

N : Node 개수

... : 임베딩 벡터 모양

B : Batch Size

num_actions: 액션 수

I. MCTX 트리구조 분석하기

```
@chex.dataclass(frozen=True)
class Tree(Generic[T]):
    """State of a search tree.
    node_visits: chex.Array # [B, N]
    raw_values: chex.Array # [B, N]
    node_values: chex.Array # [B, N]
    parents: chex.Array # [B, N]
    action_from_parent: chex.Array # [B, N]
    children_index: chex.Array # [B, N, num_actions]
    children_prior_logits: chex.Array # [B, N, num_actions]
    children_visits: chex.Array # [B, N, num_actions]
    children_rewards: chex.Array # [B, N, num_actions]
    children_discounts: chex.Array # [B, N, num_actions]
    children_values: chex.Array # [B, N, num_actions]
    embeddings: Any # [B, N, ...]
    root_invalid_actions: chex.Array # [B, num_actions]
    extra_data: T # [B, ...]
```

B : Batch Size

N : Node

개수

B : Batch Size

N : Node 개수

num_actions: 액션 수

B : Batch Size

N : Node 개수

... : 임베딩 벡터 모양

B : Batch Size

num_actions: 액션 수

Node 개수

가 있는 데이터는
전부 Node 정보

하지만 Jax 기본
데이터는 SoA
구조이므로, 전부
Array로 펼쳐야
하므로, 해당
내용처럼 표현해야
함

II. 새로 만들기



Xstructure

A Python package providing JAX-optimized data structures, including a batched priority queue and a cuckoo hash table.

Backend: SoA arrays for JAX

- Each field is declared with a `FieldDescriptor`, so the decorator stack can materialise a dedicated JAX array for the field when you call `default`, `random`, or any helper.
- Utilities such as `xnp.concat`, `HashTable.parallel_insert`, or `BGPQ.insert` work through `jax.tree_util.tree_map`, keeping all operations in the batched array world that JIT compilation, fusion, and vectorisation expect.

Interface: AoS ergonomics for users

- Indexing (`state[0]`), `.at` updates, and container APIs (`Queue.dequeue`, `Stack.pop`) rewrap the mutated arrays into the original dataclass type, so you interact with plain Python objects.
- Nested dataclasses follow the same rule, allowing deeply structured states to feel idiomatic while preserving consistent SoA storage beneath.

개인
프로젝트
결과물 활용

II. 새로 만들기

mctx 와 비슷한 데이터
구성을 가짐

하지만 AoS 구조처럼
프로그래밍

```
@xstructure_dataclass
class Tree(Xstructurable):
    root_invalid_actions: FieldDescriptor[jnp.uint8, action_shape]
    nodes: FieldDescriptor[node_class, (max_nodes,)]
    next_free_idx: FieldDescriptor[jnp.int32, (), 1]
```



```
@xstructure_dataclass
class Node(Xstructurable):
    state: FieldDescriptor[embedding_state_class] # state of the node / shape: (... , state_shape)
    visit_count: FieldDescriptor[jnp.int32, (), 0] # number of times the node has been visited / shape: ()
    parent_idx: FieldDescriptor[jnp.int32, (), NO_PARENT] # index of the parent node / shape: ()
    action_from_parent_idx: FieldDescriptor[jnp.int32, (), NO_PARENT] # action index taken to reach the node at this node at parent node / shape: ()
    children_idx: FieldDescriptor[jnp.int32, children_shape, UNVISITED] # indices of the child nodes / shape: (children_shape)
    children_prior_logits: FieldDescriptor[jnp.float32, children_shape, 0.0] # prior logits of the child nodes / shape: (children_shape)
    children_rewards: FieldDescriptor[jnp.float32, children_shape, 0.0] # rewards of the child nodes / shape: (children_shape)
    children_visits: FieldDescriptor[jnp.int32, children_shape, 0] # visits of the child nodes / shape: (children_shape)
    children_discounts: FieldDescriptor[jnp.float32, children_shape, 0.0] # discounts of the child nodes / shape: (children_shape)
    children_values: FieldDescriptor[jnp.float32, children_shape, 0.0] # values of the child nodes / shape: (children_shape)
    raw_value: FieldDescriptor[jnp.float32, (), 0.0] # raw value of the node / shape: ()
    value: FieldDescriptor[jnp.float32, (), 0.0] # cumulative search value of the node / shape: ()
```

Tree 안에 모든 Array 를 사전 설정하는것이 아닌, Node 를 따로
정의하고 Node 를 max_nodes 만큼 생성하는것

III. 간단 테스트

```
Choose your move [0, 1, 2, 3, 4, 5, 6, 7, 8]: 8
You played 8
. . .
. . .
. . X

AI played 7
. . .
. . .
. O X
```

```
Choose your move [0, 1, 2, 3, 4, 5, 6]: 5
You played 5
. . .
. . X
. O X

AI played 2
. . O
. . X
. O X
```

```
Choose your move [0, 1, 3, 4, 6]: 4
You played 4
. . O
. X X
. O X

AI played 0
O . O
. X X
. O X
```

```
Choose your move [1, 3, 6]: 3
You played 3
O . O
X X X
. O X

You win!
```

틱택토를 아주 간단히
짜서 테스트 진행

아주 약하지만

어거지로 돌아는 간다?
수준

아직 개선할게 많음...